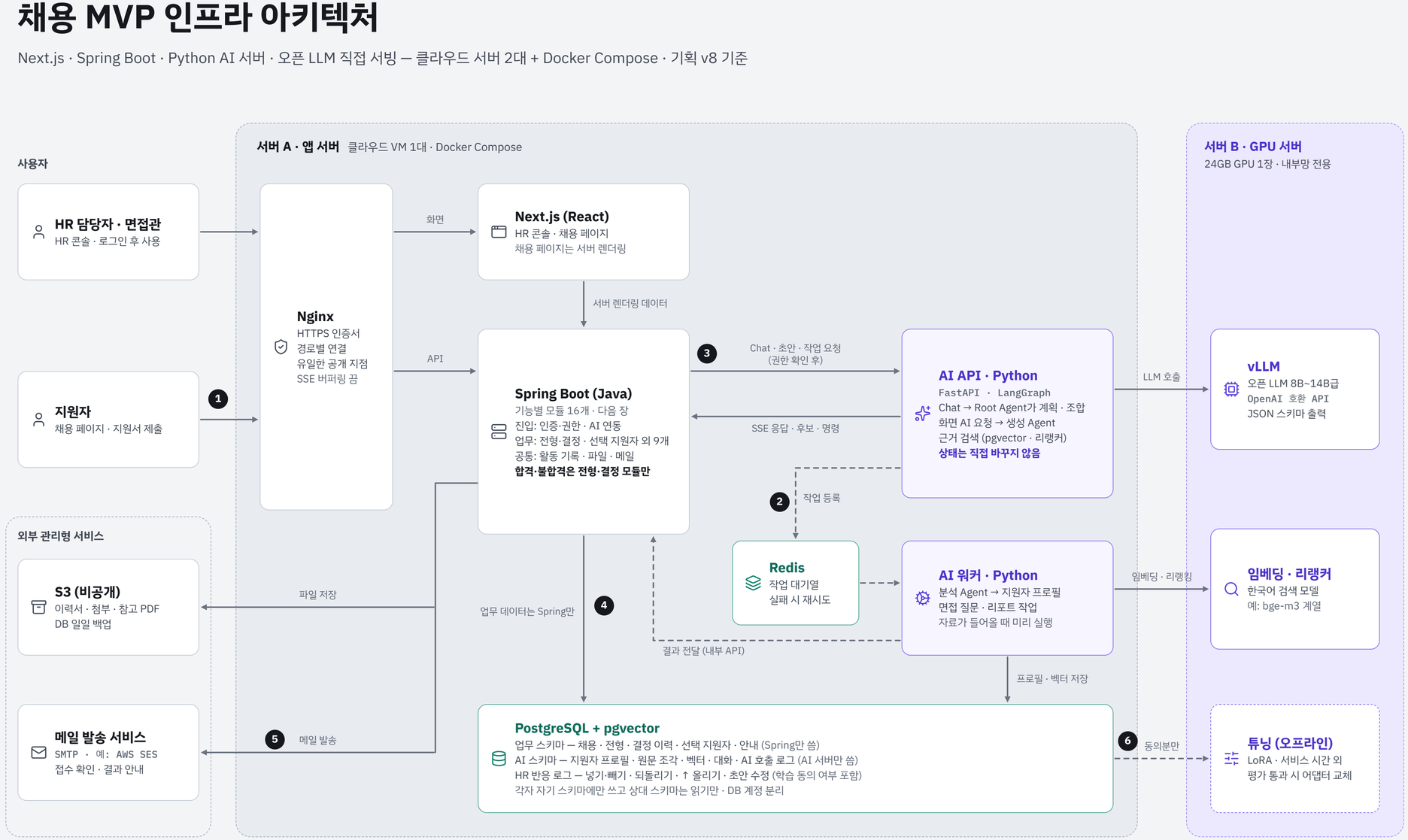
# 채용 MVP 인프라 아키텍처

Next.js · Spring Boot · Python AI 서버 · 오픈 LLM 직접 서빙 — 클라우드 서버 2대 + Docker Compose · 기획 v8 기준




## 꼭 지킬 규칙

**1. AI는 채용 결과를 바꾸지 않아요**

AI 서버는 프로필 · 분석 결과만 저장하고, 탐색 결과와 HR 지시는 '후보 · 명령'으로만 돌려줘요. 반영 · 이력 · 되돌리기는 Spring이, 합격 · 불합격은 HR 버튼으로만 해요.

**2. 밖에서 열리는 건 Nginx뿐이에요**

AI 서버 · GPU 서버 · DB · Redis는 내부망에만 둬요. 지원자 파일은 비공개 S3에 두고 서명 URL로만 열며, 지원자 자료를 외부 AI 서비스로 보내지 않아요.

**3. 분석은 자료가 들어올 때, 기록은 처음부터**

지원 · 평가 · 면접 자료가 들어오면 대기열로 미리 분석하고, AI 호출과 HR 반응을 학습 동의 여부와 함께 남겨요.

## 주요 흐름

1. **지원**
   지원자가 지원하면 Spring이 PDF를 S3에 저장하고 접수 확인 메일을 보낸 뒤, AI 서버에 분석 작업을 요청해요.
2. **미리 분석**
   AI 워커의 분석 Agent가 지원자 프로필(근거 · 출처)과 서류 점수를 만들고, 점수 · 경력은 Spring에 넘겨요. Chat은 이 프로필을 먼저 찾아서 빨라요.
3. **Chat 탐색**
   HR 질문 → Spring(권한 확인 · 컨텍스트) → Root Agent가 찾기는 프로필 검색 · 비교로, 분석은 도구 루프로 처리. 답은 SSE로 흘려 보내요.
4. **선택과 결정**
   AI가 돌려준 후보와 넣기 · 빼기 명령은 Spring이 검증해 선택 지원자에만 반영해요. 합격 · 불합격은 HR이 체크 후 버튼으로 하고 이력 · 되돌리기가 남아요.
5. **안내 메일**
   결과 확정 뒤 Communication Agent 초안을 HR이 확인 · 수정하면, Spring이 변수를 채워 메일 서비스로 보내요.
6. **튜닝**
   학습에 동의한 로그를 가명 처리해 서비스 시간 외에 LoRA 튜닝 → 평가셋 비교 → 통과하면 vLLM 어댑터 교체.

## 서버 · 배포

**서버 A** 일반 VM 4 vCPU · 16GB (예: AWS t3.xlarge 급). Nginx · Next.js · Spring · AI API · AI 워커 · PostgreSQL · Redis

**서버 B** GPU 24GB 1장 (예: AWS g6 · g5 계열). 개발 · 시연 시간에만 켜고, 꺼진 동안 쌓인 분석 작업은 켜지면 이어서 처리해요.

**모델** 한국어 평가셋으로 8B~14B급 오픈 모델 2~3개를 비교해 골라요. 14B급은 4bit · FP8 양자화로 올리고, 상업 이용 라이선스를 확인해요.

**배포** GitHub Actions로 이미지를 빌드하고, 두 서버에서 docker compose로 실행해요. DB 스키마는 Spring(Flyway)과 AI 서버(Alembic)가 따로 관리해요.

## AI가 멈춰도

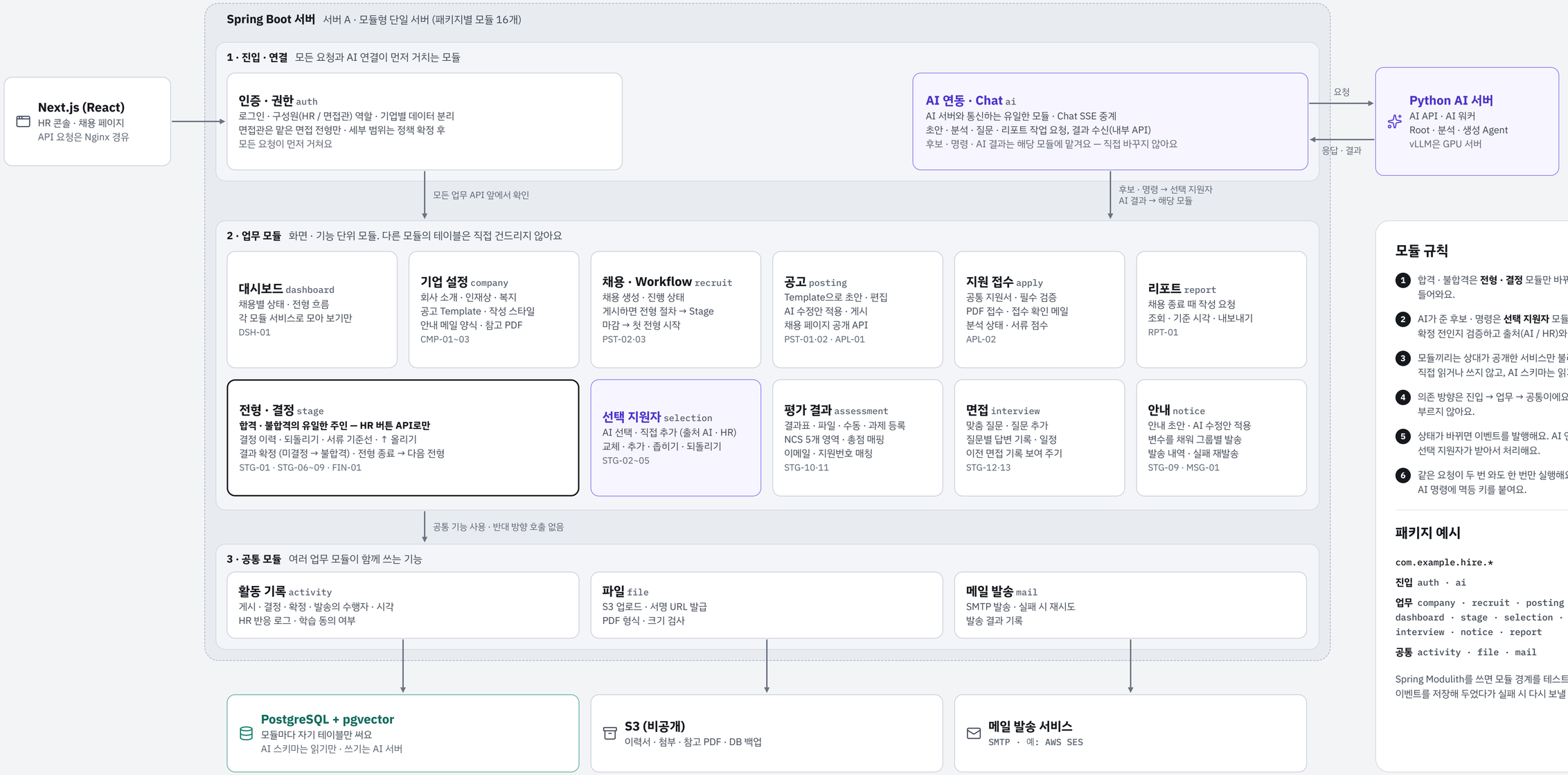
GPU 서버가 꺼지거나 AI 서버가 멈춰도 공고 · 결정 · 확정 · 발송은 Spring만으로 계속돼요. 분석 작업은 대기열에 남았다가 다시 처리되고, Chat은 잠시 뒤 다시 묻도록 안내해요.

## 일부러 안 쓰는 것

쿠버네티스 · 마이크로서비스 · 별도 벡터 DB · Kafka. MVP 규모에선 운영 부담만 커요. Spring 안에서 모듈만 잘 나눠 두면 나중에 분리할 수 있어요.

# Spring Boot 기능별 모듈

서버 A의 Spring 서버 1개를 기능별 패키지(모듈)로 나눈 구조 — 경계를 지키면 나중에 모듈 단위로 떼어낼 수 있어요




## 모듈 규칙

1. 합격 · 불합격은 **전형 · 결정** 모듈만 바꿔요. HR의 버튼 API로만 들어와요.
2. AI가 준 후보 · 명령은 **선택 지원자** 모듈만 바꿔요. 대상 · 권한 · 확정 전인지 검증하고 출처(AI / HR)와 되돌리기를 남겨요.
3. 모듈끼리는 상대가 공개한 서비스만 불러요. 다른 모듈의 테이블은 직접 읽거나 쓰지 않고, AI 스키마는 읽기만 해요.
4. 의존 방향은 진입 → 업무 → 공통이에요. 공통 모듈은 업무 모듈을 부르지 않아요.
5. 상태가 바뀌면 이벤트를 발행해요. AI 연동 · 활동 기록 · 대시보드 · 선택 지원자가 받아서 처리해요.
6. 같은 요청이 두 번 와도 한 번만 실행해요. 지원 · 결정 · 확정 · 발송 · AI 명령에 멱등 키를 붙여요.

## 패키지 예시

`com.example.hire.*`

**진입** `auth` · `ai`

**업무** `company` · `recruit` · `posting` · `apply` · `dashboard` · `stage` · `selection` · `assessment` · `interview` · `notice` · `report`

**공통** `activity` · `file` · `mail`

Spring Modulith를 쓰면 모듈 경계를 테스트로 검사하고, 발행한 이벤트를 저장해 두었다가 실패 시 다시 보낼 수 있어요 (선택).

# AI Agent 구조

실행 시점 3가지 · Root Agent + 하위 Agent 6개 (Interview Agent는 분석 · 생성 두 역할) · 정해진 일은 코드로, 판단은 LLM으로 · Chat의 B 분석 · 생성만 도구 루프 — 기획 v8 §11~13

Root Agent · 분석 Agent · 생성 Agent
코드 · 도구 (LLM 없음) · 데이터 · 코드 LLM 처리 단계

## ① 자료가 들어올 때 · AI 워커가 분석 Agent를 미리 실행 (비동기)

**자료 도착**
지원 접수
평가 결과 등록
면접 답변 기록
Spring이 작업 요청

→

**AI 워커**
대기열에서 꺼내
알맞은 Agent 실행
같은 자료는 한 번만

→

분석 Agent · Chat에서는 후보 근거 재확인에도 쓰임

코드 ① 원문 분할 · 위치 보존 → LLM ② 항목 추출 (JSON) → 코드 ③ 근거 원문 대조 → ↻ 실패 항목만 다시 · 최대 2회 → 코드 ④ 임베딩 · 저장

**Application Agent**
시점 지원 접수 때
입력 이력서 · 자기소개서 · 포트폴리오 · 자격증 공고 요건 · 인재상
결과 경험 · 역량 · 자격 조건 + 근거 문장
**서류 점수** (자격 요건 · 우대 사항 기준)

**Assessment Agent**
시점 평가 결과 등록 때
입력 NCS 5개 영역 점수 · 총점 · 결과 파일 과제 제출물 · URL
결과 점수 · 등급은 코드로 그대로 저장 제출물의 강점 · 약점 + 근거

**Interview Agent** 결과 분석
시점 질문별 답변 기록 때
입력 맞춤 질문 · 답변 기록 이전 면접의 확인 포인트
결과 확인된 점 · 아직 확인되지 않은 점 상충하는 점 + 근거

→

**지원자 프로필 DB**
지원자별로 전형마다 쌓임
항목마다 근거 문장 · 출처
서류 점수 · 경력은 Spring에도
`PostgreSQL + pgvector`

## ② AI Chat 질문 · Root Agent가 해석하고, 필요한 정보와 Agent를 골라 조합

**AI Chat 질문**
현재 채용 · 현재 전형
@멘션 · 체크한 지원자
자연어 질문
Spring이 권한 확인 후 전달

→

**Root Agent**
Recruitment Orchestrator

① **대상 확인**
전체 · @멘션 · 체크한 지원자

② **요청 유형 판단**
A 찾기 · B 분석 · 생성 · C 편집

③ **실행 계획**
A · C는 JSON 계획 · B는 도구 루프

④ **결과 정리**
한 응답으로 · SSE 스트리밍

응답 필드
`answer · mode · candidates · command`

차별 조건 요청은 여기서 차단
직접 분석하거나 상태를 바꾸지 않음

→

**A · 지원자 찾기**
"~ 찾아줘" → 교체
"~도 추가해줘" → 추가
"그중 ~" → 좁히기

→ **프로필 검색** 코드
조건 + 의미 검색 → 리랭커
비교는 코드로 · 같은 질문엔 같은 결과
서류 전형은 기준 통과자만

→ **후보 근거 재확인** 상위 후보 · 병렬
Application · Assessment · Interview · 분석
원문 근거가 조건을 받치는지 판정

→ **선택 지원자 반영**
리스트 테두리 · Chat 요약
Spring이 저장 · 출처 AI

**B · 분석 · 생성**
지원자 의견 · 비교 · 현황
면접 질문 생성

→ **도구 루프** LLM
조회 도구를 골라 부르고
결과를 보고 다음 자료를 정해요
프로필 · 원문 · 전형 결과 · 회사 소개

→ **전문 Agent 호출** 필요할 때 · 도구로
Application · Assessment · Interview · 분석
Interview · 질문

→ **대화로 답변**
근거 · 출처(글) · 미확인 구분
면접 질문은 카드에 저장
모델 호출 상한 안에서 끝냄

**C · 선택 지원자 편집**
넣기 · 빼기 · 비우기 · 되돌리기
하위 Agent를 거치지 않음

→ **{ } 명령(JSON)으로만 바꿈** 대상은 코드가 확정
동작 · 대상 id · 멱등 키만 담아요. 대상은 @멘션 · 체크 · 이름으로 코드가 정해요.
0명 · 동명이인이면 실행하지 않고 되물어요.
**합격 · 불합격 요청은 실행하지 않고** '체크 후 맨 아래 버튼'으로 안내해요.

→ **Spring 검증 · 실행**
선택 지원자 반영 · 출처 HR
처리한 이름 알림

## ③ 화면 AI 요청 · 채용 이벤트 · Root Agent를 거치지 않고 생성 Agent를 바로 실행

**화면 · 이벤트**
공고 AI 수정 요청
면접 전형 시작
안내 초안 · 수정 요청
채용 종료

→

생성 Agent · Chat으로 요청하면 Root Agent가 부름 만들고 → 코드로 검사 → 필요하면 1회 고침

**Posting Agent**
시점 공고 편집기의 AI 수정 요청 Template 채우기는 코드
입력 HR 요청 · 현재 공고 작성 스타일 · 기존 공고
결과 바뀐 부분만 수정안 전형 이름 + 서류 · 평가 · 면접 필수 제출 서류 · 애매하면 되물음

**Interview Agent** 질문 생성
시점 면접 전형 시작
입력 지원자 프로필 서류 · 평가 · 이전 면접 기록
결과 확인 안 된 부분 중심의 질문 문장 만든 이유는 Chat에서 설명

**Communication Agent**
시점 결과 확정 뒤 안내 초안 · 수정 요청
입력 확정 결과 · 안내 메일 양식 4종
결과 AI 작성 영역만 채운 초안 필수 문구 · 변수 자리는 고정

**Report Agent**
시점 채용 종료 때 · 비동기 작업
입력 전형별 집계 · 결정 이력 지원자 프로필
결과 문서형 최종 리포트 · 기준 시각 숫자는 집계값 그대로 채용 상태는 바꾸지 않음

→

**초안 · 질문 · 리포트**
HR이 확인 · 수정 · 적용한 뒤 게시 · 발송
**AI는 게시 · 발송하지 않음**

## 공통 기반 · 모든 Agent가 함께 씀

**지원자 프로필**
지원자별로 전형마다 쌓이는 정리 정보
항목마다 근거 문장 · 출처 · 확인 상태

**검색 도구**
조건 검색 — 경력 · 점수 · 결정 상태
의미 검색 — 문장 유사도 (pgvector) → 리랭커

**근거와 출처**
모든 답변 · 선택 결과에 근거 문장 · 출처
출처는 글로 표시 · 근거가 없으면 '미확인'

**범위 · 가드레일**
회사 · 채용 · 전형 범위는 코드가 넣음
지원서 속 지시문은 데이터로만 · 반복 상한

**차별 조건 차단**
나이 · 성별 · 출신 지역 · 혼인 여부 · 신체 조건
탐색 요청은 실행하지 않고 안내 · 범위는 법무 검토

**한 모델 · 역할별 지시문**
Agent마다 모델을 따로 띄우지 않음
지시문 · 도구 · JSON 출력 형식만 다르게

**상태 변경은 Spring만**
AI는 후보 · 명령 · 초안까지
서버가 검증 · 실행 · 이력 기록